2025-2026

Welcome to the 11th Lab session of CO3519 module (Semester 02)

For Two Hours Lab Session:

#	Task	Time (m)	Guidance
A	Please explore the concept about the Perceptron and go through the DL history quickly.	20	You can think and contextualise the concepts towards implementation.
	Please implement/run the code for simple neural network (perceptron).	30	Please follow the instruction on Page 2 to Page 4. Please execute the step 1 and sub-step 1.1.
	Break	10	
B	Please continue with the second step on the Page 4.	50	The second step will help you how to develop your own NN for different task. Build your deep understanding.

For One Hour Lab Session:

#	Task	Time (m)	Guidance
1	Please perform the task **A** from two hours sessions.	50	While doing task A, two sub-tasks such as exploration of concept and then the implementation of NN.

Implementing Simple Neural Network

1. Development of simple NN with example.

Introduction to Perceptrons

A Perceptron is a type of artificial neuron used in machine learning. It simulates the way a single biological neuron processes inputs and produces an output. The perceptron is the simplest form of a neural network.

Key Components of a Perceptron:

Inputs (Features): Data fed into the model. In this example, we'll use hours studied and hours slept.

Weights: The importance assigned to each input.

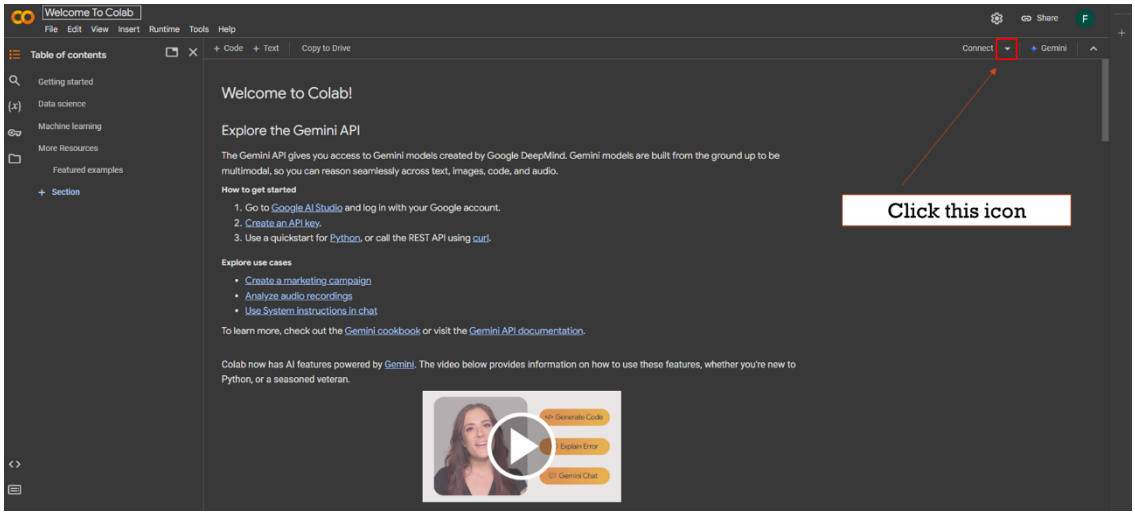
Bias: Adjusts the output to make better predictions.

Output: The perceptron's prediction (e.g., "Pass" or "Fail").

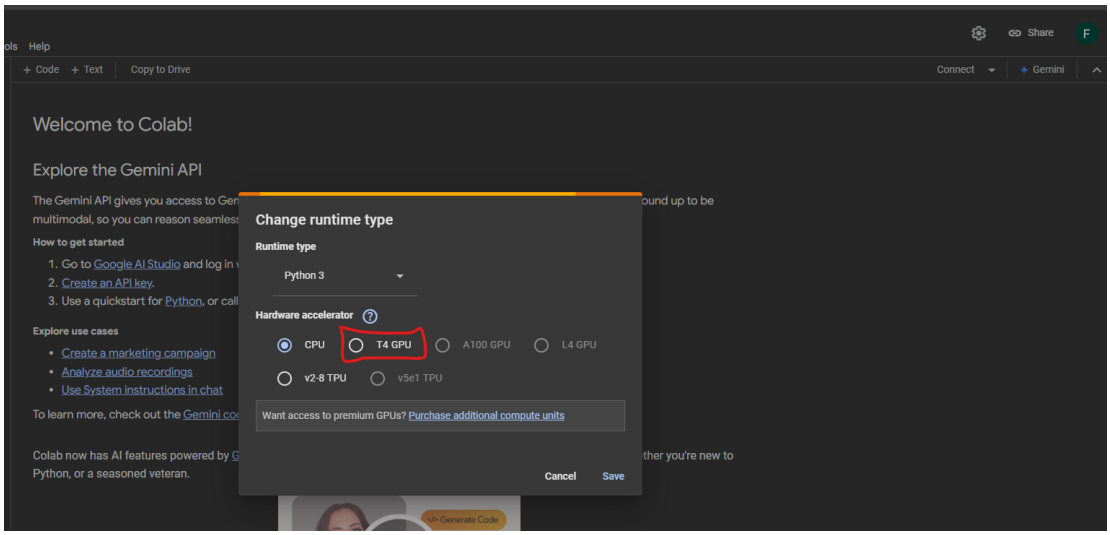
Example: Pass or Fail Prediction.

Please for this description, open the lecture PPT.

Before starting the implementation, make sure to set the environment as follow and keep this setting throughout this semester.



In the dropdown, click "Change runtime type" then following screen will appear:



Here, please the **T4 GPU** or another supporting GPU and then save.

The function perceptron () calculates the weighted sum of the inputs and applies the step function to determine if the student passes (1) or fails (0).

Inputs: Hours studied and hours slept.

Weights: Importance assigned to the inputs.

Bias: Adjusts the prediction.

```
[ ] # Perceptron Function
    def perceptron(inputs, weights, bias):
        weighted_sum = sum(x * w for x, w in zip(inputs, weights)) + bias
        return 1 if weighted_sum >= 0 else 0

    # Inputs and Parameters
    inputs = [4, 3]  # Hours studied and slept
    weights = [0.6, 0.4]
    bias = -0.5

    # Prediction
    output = perceptron(inputs, weights, bias)
    print("Prediction (Pass = 1, Fail = 0):", output)

Prediction (Pass = 1, Fail = 0): 1
```

Now, let's visualize how the perceptron separates the "Pass" and "Fail" regions based on the given inputs.

We will plot the decision boundary, where the perceptron makes its decision.

```
[ ] import numpy as np
    import matplotlib.pyplot as plt

    # Parameters
    weights = np.array([0.6, 0.4])
    bias = -0.5

    # Decision boundary: w1*x1 + w2*x2 + b = 0
    x1 = np.linspace(0, 10, 100)
    x2 = -(weights[0] * x1 + bias) / weights[1]  # Solve for x2

    # Plot data points
    plt.plot(x1, x2, label="Decision Boundary")
    plt.fill_between(x1, x2, 10, color="lightblue", alpha=0.5, label="Pass Region")
    plt.fill_between(x1, x2, 0, color="lightcoral", alpha=0.5, label="Fail Region")
    plt.scatter(4, 3, color='green', label="Student (4 hrs study, 3 hrs sleep)")
    plt.xlabel("Hours Studied")
    plt.ylabel("Hours Slept")
    plt.title("Perceptron Decision Boundary")
    plt.legend()
    plt.show()
```

Decision Boundary: The line where the perceptron switches its decision between pass and fail.

Blue Region: Represents where students pass.

Red Region: Represents where students fail.

Green Dot: Represents the specific student (4 hours studied, 3 hours slept) we predicted.

1.1. Please perform the following tasks.

Modify the Weights and Bias:

Change the values of weights and bias, and observe how the decision boundary and predictions change. What happens when the bias is set to 0?

Experiment with Different Features:

Add more features like previous exam scores or attendance percentage, and modify the perceptron code to work with multiple features.

Modify the rule for passing (e.g., increase the threshold for hours studied and slept).

2. Task: Develop NN

Use the above knowledge and develop your own NN/Perceptron for the following different problems (3, 4, 5). You can check different parameters, get help from different sources. Next, I have solved and developed the perceptron for (1 & 2) and its link is given there.

1. Loan Approval Prediction

   ➔ Predict whether a loan is *Approved or Rejected*

   Code: https://colab.research.google.com/drive/1xWyWVbRJ_tZucMS8TcZWYdXG4TIRkFMR?usp=sharing

2. Email Spam Detection

   ➔ Predict whether an email is *Spam or Not Spam*

   Code: https://colab.research.google.com/drive/1xWyWVbRJ_tZucMS8TcZWYdXG4TIRkFMR?usp=sharing

3. Weather-Based Decision (Play or Not Play)

   ➔ Decide whether to *Play Outside* or *Not Play*

4. Employee Performance Classification

   ➔ Predict whether employee performance is *Good* or *Poor*

5. Disease Risk Prediction (Basic)

   ➔ Predict whether a patient is *At Risk* or *Not at Risk.*